Introducción a IoT

TECNOLOGÍAS PARA LA AUTOMATIZACIÓN

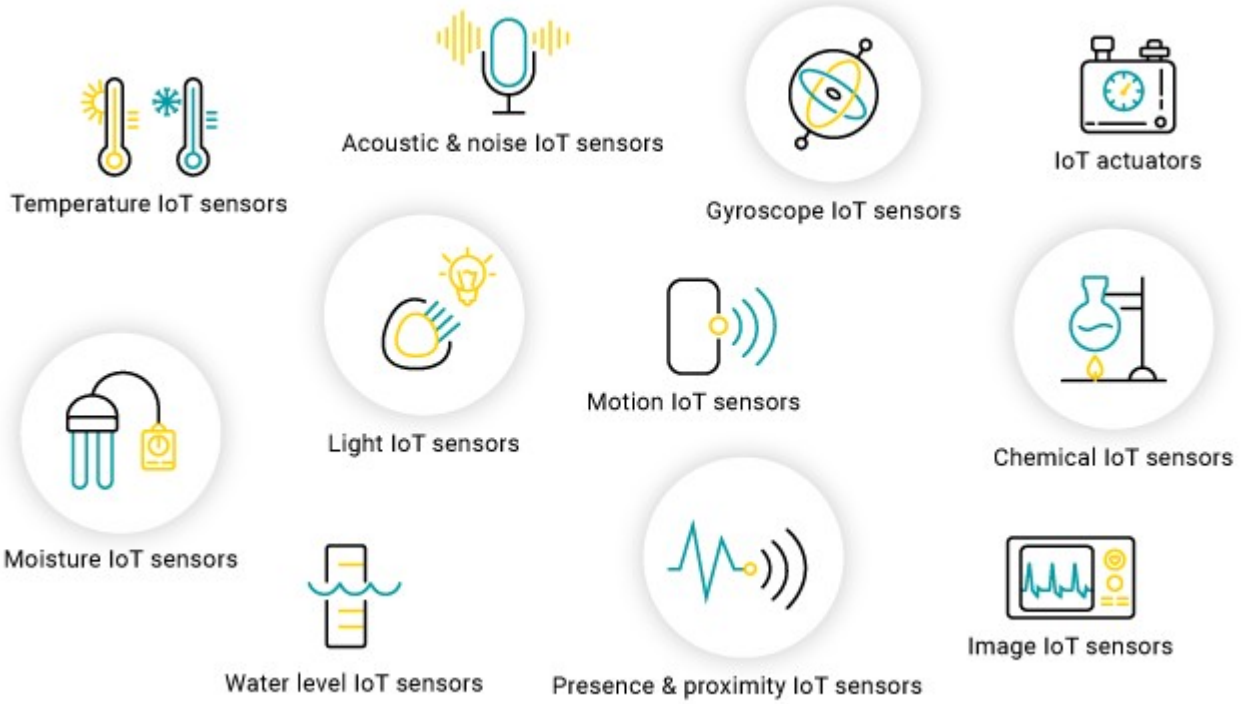


IoT (Internet of Things)

Los **dispositivos IoT** se conectan con un proceso llamado **M2M** (machine to machine) en el que **dos dispositivos** o máquinas cualesquiera **se comunican entre sí** utilizando cualquier tipo de conectividad (que puede ser **cable, WiFi, Bluetooth, etc**.), haciendo su trabajo sin la necesidad de que un humano intervenga.

Similar al **Peer to Peer** de los ordenadores para jugar online o compartir archivos.

Un dispositivo IoT **recolecta, procesa y analiza** una gran cantidad de información, entre la que se incluye información sobre los **hábitos y preferencias** de consumo de cada uno de los usuarios, así como por ejemplo poder gestionar su **salud u otras utilidades** para el día a día.
Como, por ejemplo, si hay algún fallo en tu coche, el ordenador a bordo del mismo puede enviar un aviso para que lo lleves al taller.



IoT (Internet of Things)

Un dispositivo IoT consiste en un **objeto** al que se le ha **dotado de conexión a Internet** y cierta **inteligencia** de software, al que se puede acceder para medir parámetros físicos (mediante **sensores**) o actuar remotamente (**actuadores**) y que por tanto permite generar un ecosistema de servicios alrededor del mismo.

Un ejemplo de dispositivo IoT es el altavoz inteligente Alexa de Amazon, el cual se encuentra conectado a la red de redes para obtener información e interactúa con otros dispositivos para darnos la capacidad de controlarlos a través de simples comandos de voz.

Altavoces inteligentes
Luces
Enchufes
Termostatos
Camaras de seguridad
Televisores
Electrodomésticos (lavadoras, secadoras, hornos, aspiradoras)

Para lograr que sensores y actuadores recolecten y envíen datos del mundo físico a Internet, los dispositivos IoT cuentan con un componente esencial: **el microcontrolador**.

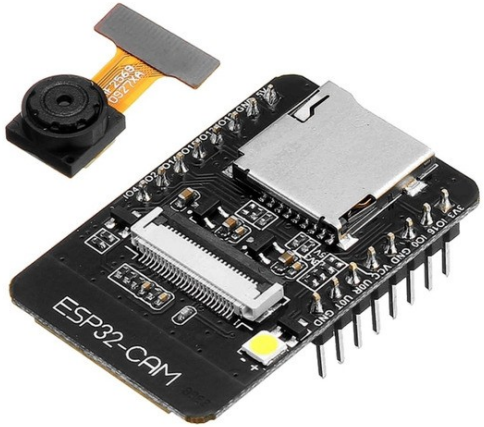
Los microcontroladores son **sistemas de control embebidos**, diseñados para realizar **funciones dedicadas**, normalmente en **tiempo real**.

Ordenadores → de propósito general, → amplio rango de necesidades
Sistemas embebidos → necesidades específicas → multitud de productos en los que se requiere una cierta potencia de proceso como electrodomésticos, vehículos, máquinas, etc.

Es una tarea multidisciplinaria, Electrónica y Sistemas deben colaborar para completar la tarea.

ESP32 (Y su primo más famoso)

- Fácil para principiantes
- Comunidad activa y de código abierto
- Amplia compatibilidad de hardware y software
- Bajo costo
- Versatilidad (IoT, Robótica, monitoreo Ambiental)









Hace no tantos años, diseñar un sistema electrónico que combinara alguna capacidad electrónica con software, **no era nada fácil**.
Coordinar las placas, diseñarlas, los lenguajes no estaban desarrollados.

Hace 25 años, se trabajaba con **Z80**, y se programaba en **Assembler**.

El micro por un lado, memoria RAM → OSCILADOR por otro → MEMORIA DEL PROGRAMA.

Borrar la memoria con luz **ultravioleta**,
ESPERAR que se borre,
VERIFICAR que se haya borrado,
GRABAR el programa,
VERIFICAR que se haya grabado correctamente…,
programando en código máquina en hexadecimal.

Hay varios desarrollos independientes, con la aparición de Arduino esto explota, provocando una estandarización de facto.
Arduino es una plataforma de creación de electrónica de **código abierto**, la cual está basada en **hardware y software libre**, flexible y fácil de utilizar para los creadores y desarrolladores.

Procesador ATMega328P.
Velocidad de reloj: 16 MHz.
Voltaje de trabajo: 5V.

El **hardware libre** son aquellos dispositivos cuyas **especificaciones y diagramas** son de **acceso público**

Ventajas:

Qué nos solucionó Arduino

- Disponemos de HW que funciona.
- Construir HW es caro y lleva mucho tiempo.
- Evitamos las 3 etapas de diseño:
  - Conceptual
  - Desarrollo de ingeniería
  - Desarrollo final.
- Se pueden agregar fácil y rápidamente otras capacidades al HW.
  - Red Ehernet
  - Displays
  - Sensores
  - Bluetooth
- Esquemático disponible (circuito eléctrico)

Por ejemplo, si se quisiera conectar tu Hw con una red ethernet, hubiera sido necesario **buscar un chip** controlador que se comunique con ethernet, **desarrollar una placa** alrededor (no se puede trabajar directamente sobre el chip), **conectarle tu microcontrolador**, te **equivocás**, lo haces de nuevo, hay que **programarlo**, entonces hay que ver el **datasheet** o manual del controlador ethernet, e ingeniárselas para sacar un byte de ese dispositivo.

Con arduino, alguien ya lo hizo, te vende el HW, lo **integrás** y eso viene con una **librería**, donde ahora conectarte con **ethernet** es simplemente **"Ethernet.Begin"**. Y todo lo que pasa por debajo... te olvidás.

Comienzan a aparecer, otras morfologías , como **galileo** de Intel, que quisieron competir pero fueron quedando en el camino.

Arduino y todo lo compatible, siguió **creciendo** y evolucionando. En esta evolución es que aparece la placa que vamos a utilizar en este curso, la **ESP32**.

Algo muy importante es que, si uno quiere diseñar un producto nuevo, no debe adaptar el HW al diseño de esta placa.
Lo genial es que el esquemático (el circuito eléctrico) también está disponible (es público) para un software de diseño (por ejemplo Eagle que es gratuito) ,
Entonces el fabricante de la placa pone a disposición el archivo donde está el diseño de esa placa, entonces lo puedo probar el diseño por software antes de fabricarlo.
Uno se ahorra el tener que homologar el producto y certificarlo, por ejemplo no va a interferir con un celular o un marcapaso, etc.

Por ejemplo, compro un magnetómetro, tengo su diseño y lo conecto con mi ESP, por medio de software "conecto" las dos piezas de HW y pruebo si funciona correctamente. Diseño conceptual LISTO.
Tomo el esquemático del arduino, copio y pego, tomo el esquemático de la brújula, copio y pego, conecto, pruebo, y tengo un diseño integrado que puedo enviar al fabricante chino para que produzca.
Entonces ahora cualquiera puede hacer un desarrollo para automatizar algo, podemos ponerle un microprocesador a TODO, esto es IoT

¿Qué es el ESP32?

ESP32 es un microcontrolador de bajo costo y bajo consumo de energía desarrollado por Espressif Systems.

Es un SoC (System on Chip)

Ofrece conectividad Wi-Fi y Bluetooth, así como procesamiento de datos de alta velocidad y capacidades de almacenamiento.

Por eso la cátedra lo eligió para desarrollar proyectos de IoT.

Arquitectura de la ESP32



- Procesador Tensilica Xtensa LX6 de doble núcleo de 32 bits de hasta 240 MHz.
- Regulador de tensión
- Unidad de procesamiento de señal digital (DSP)
- Unidad de aceleración de cifrado.

Ambos núcleos pueden trabajar de manera **independiente** o en **conjunto** para aumentar el rendimiento.

El voltaje es de **3,3 V** en comparación con el voltaje de funcionamiento de Arduino de **5 V**.
Conectado a la red eléctrica no habrá diferencia en el consumo de energía.
Alimentadas con baterías, las placas basadas en ESP tendrán un tiempo de ejecución más prolongado, ya que estas placas podrían funcionar a 4V (típico de cuando una batería decae), mientras que a 4V las placas Arduino se apagaron.
Cada PIN da acceso al exterior (al mundo).

DSP: Puede realizar operaciones de filtrado digital, como filtros de paso bajo, paso alto, paso de banda, etc., y procesamiento de señales provenientes de otros sensores.
Permite implementar una amplia gama de **operaciones matemáticas y algoritmos en señales digitales**, lo que habilita muchas aplicaciones en áreas como **procesamiento de audio**, **comunicaciones inalámbricas** y **control** en tiempo real.

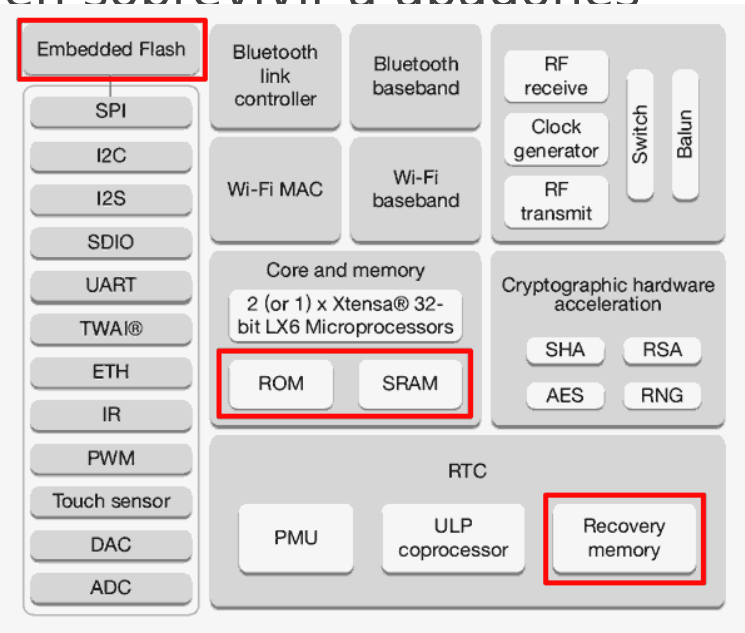
Unidad de aceleración de cifrado: diseñado específicamente para realizar **operaciones criptográficas** de manera **más eficiente y rápida** que la implementación de software pura.

Arquitectura de la ESP32

MEMORIAS

- FLASH, de hasta 4MB, para almacenar el código del programa y datos permanentes. (No volátil)
- SRAM, de hasta 520 KB, para almacenar datos temporales y programas en ejecución. (volátil)
- ROM de 448 KB, para el bootloader del sistema operativo y otros códigos esenciales de sólo lectura.
- EEPROM, se utiliza para almacenar datos que deben sobrevivir a apagones o reinicios de l

Muy útil para guardar configuraciones



Arquitectura del ESP32 DOIT

Regulador de Tensión

GPIO (General Purpose Input – Ouput) Pins

Enable

Puerto USB

Boot



ESP32

Driver USB

Como dijimos la placa ESP32 tiene incorporado un **puerto USB**, que no es nativo del chip.
Para eso tiene un chip adicional que es un **driver** para comunicarse con el protocolo del USB.
Se encarga de poner la ESP32 en **modo programación**, de cargarle el SW en la memoria y **rebootearla** (cosa que a veces hay que hacer de forma manual mediante el boton BOOT).
El botón ENABLE funciona como un Reset.

Tiene también un **regulador de tensión**, funciona en conjunto con dos de los pines de la placa, GND y VIN. Adapta el voltaje a los niveles que usa el microcontrolador y lo protege de sobrevoltajes.

GND (ground) es Tierra. VIN es un pin donde podemos dar tensión hasta 12V para independizarnos del USB.
Nosotros la **alimentación** la tomaremos desde el **USB** (entran 5V pero el chip trabaja con 3,3V)
No vamos a utilizar VIN porque la placa tiene un **error de diseño,** que si alimentamos desde VIN y conectamos algo al USB, quemamos el dispositivo.
Como la intención es utilizar siempre la conectividad USB (nos va a hacer la vida más fácil) es que resignaremos el uso del pin VIN.

Protocolos de comunicación

- **SPI** (Serial Peripheral Interface) para enviar datos entre microcontroladores. Es un bus de datos síncrono, es decir, utiliza un reloj para regular la transferencia de datos.
- **I²C** (Inter-Integrated Circuit) más utilizado para enviar y recibir datos de otros dispositivos, como pantallas OLED, sensores de presión barométrica y temperatura, etc.
- **I2S** (Inter-IC Sound), es un estándar de interfaz de bus serie eléctrico que se utiliza para conectar dispositivos de audio digital entre sí.
- **UART** (Receptor / Transmisor Asíncrono Universal) no es un protocolo de comunicación como SPI e I2C, sino un circuito físico en un microcontrolador. El objetivo principal es transmitir y recibir datos en serie.

Arquitectura de la ESP32

PERIFERICOS

- Conector micro-USB para la alimentación y comunicación
- 30 ó 36 Conectores GPIO para entrada y salida de señales digitales
- 3 interfaces serie SPI (HW específico, 2 disponibles)
- 2 interfaces I2C (HW específico)
- Interfaces I2S
- 2 canales de conversor analógico a digital de 12 bits (ADC)
- 2 canales de conversor digital a analógico (DAC)
- 16 canales de salida PWM (Pulse Width Modulation)
- 3 interfaces UART (2 disponibles)
- 9 GPIO de detección capacitiva (Touch)
- Botones ENABLE y BOOT
- Conectividad Wi-Fi y Bluetooth
- LED integrado

La ESP tiene HW que se ocupa de realizar la comunicación Serie o SPI o I2C, es decir, cuando tiene que enviar un mensaje, envía la tarea al HW y el procesador se dedica a otra cosa.
(Se puede hacer con otros pines, pero hay que bajar librerías y se ocupa tiempo del procesador).

La interfaz I2S (Inter-IC Sound) en el ESP32 (serial síncrono) diseñado principalmente para la transmisión y recepción de datos de audio digital entre dispositivos, como micrófonos, DACs (convertidores digital-analógico), ADCs, altavoces y otros periféricos de audio.

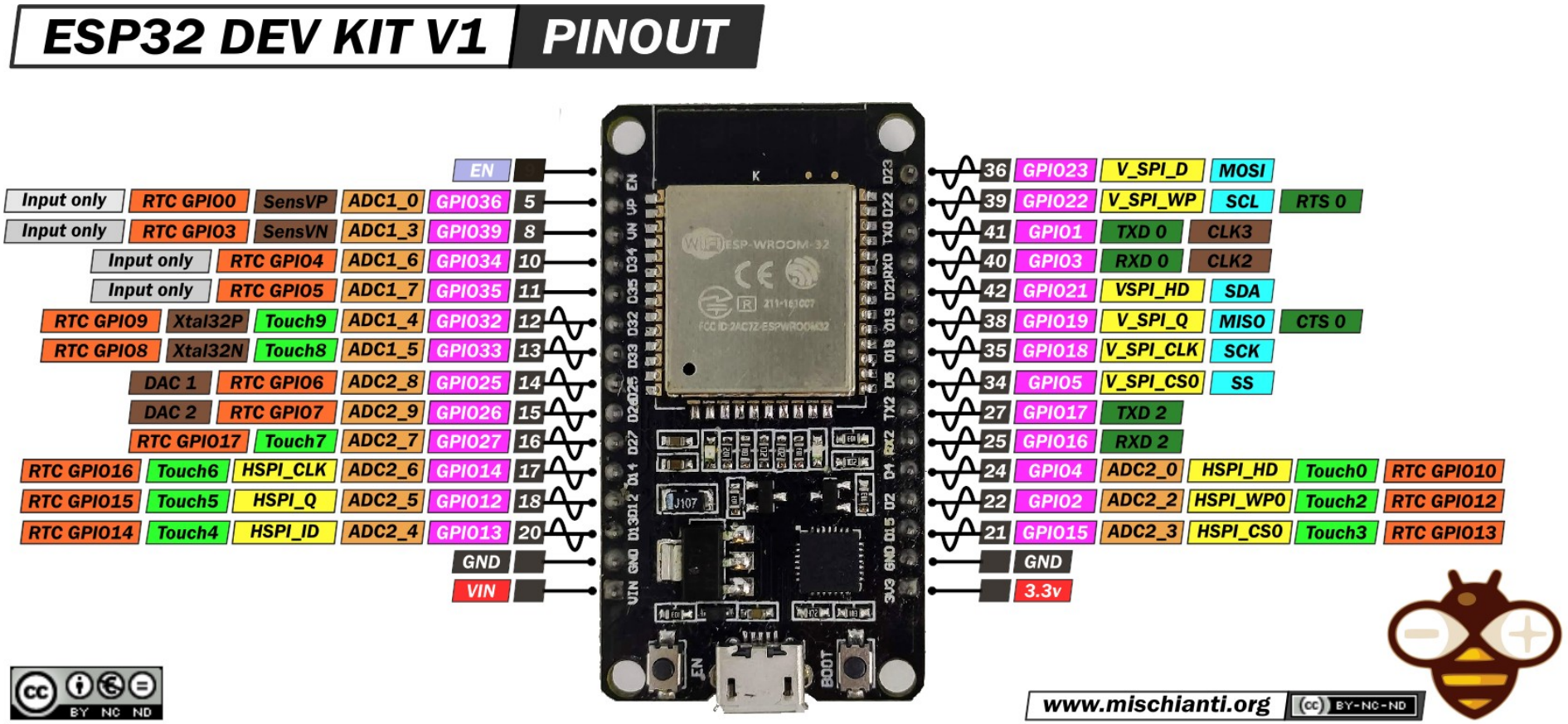
ADC se utiliza para medir una señal analógica y convertirla en una señal digital.

DAC se utiliza para generar señales analógicas continuas y precisas.

PWM (**P**ulse **W**idth **M**odulation) se utiliza para controlar la intensidad o la velocidad de una señal analógica mediante la variación de la duración y el ancho de los pulsos de una señal digital (NO es analógica continua).
Se puede usar para control de motores, LEDs, servos, generación de señales analógicas simuladas.

Touch:

- Detectan variaciones en capacitancia causadas por contacto o proximidad.
- Permiten crear botones táctiles sin partes mecánicas.
- Pueden despertar el ESP32 desde modo “deep sleep” al detectar toque.
- Se pueden conectar a objetos conductores externos para sensores personalizados.
- Son de bajo ruido y alta sensibilidad para detectar cambios sutiles.



GPIOs en detalle

Cada PIN de la placa ESP no tiene una sola función específica, sino que se puede programar para que haga distintas cosas según la necesidad.

Las entradas digitales son binarias 1 ó 0, T ó F. Se pueden conectar Pulsadores, sensores discretos (finales de carrera de impresoras 3D, interruptores).
A las salidas digitales se pueden conectar Leds, Relés.
Por SW le vamos a decir al PIN que se convierta en salida o entrada (los GPIO)
Entonces el PIN 2 es salida y el PIN11 es entrada.

El pin 3V3 sirve para alimentar otros dispositivos con 3.3V (hasta un máximo de 800 mA, usar con precaución)
Hay dos pines de alimentación de entrada y salida.

El resto de los pines se pueden usar como entradas o salidas digitales.
Para leer un 1 ó 0 (T o F).

Muchos de ellos como entrada analógica, es decir una señal que mida entre 0V y 3,3V

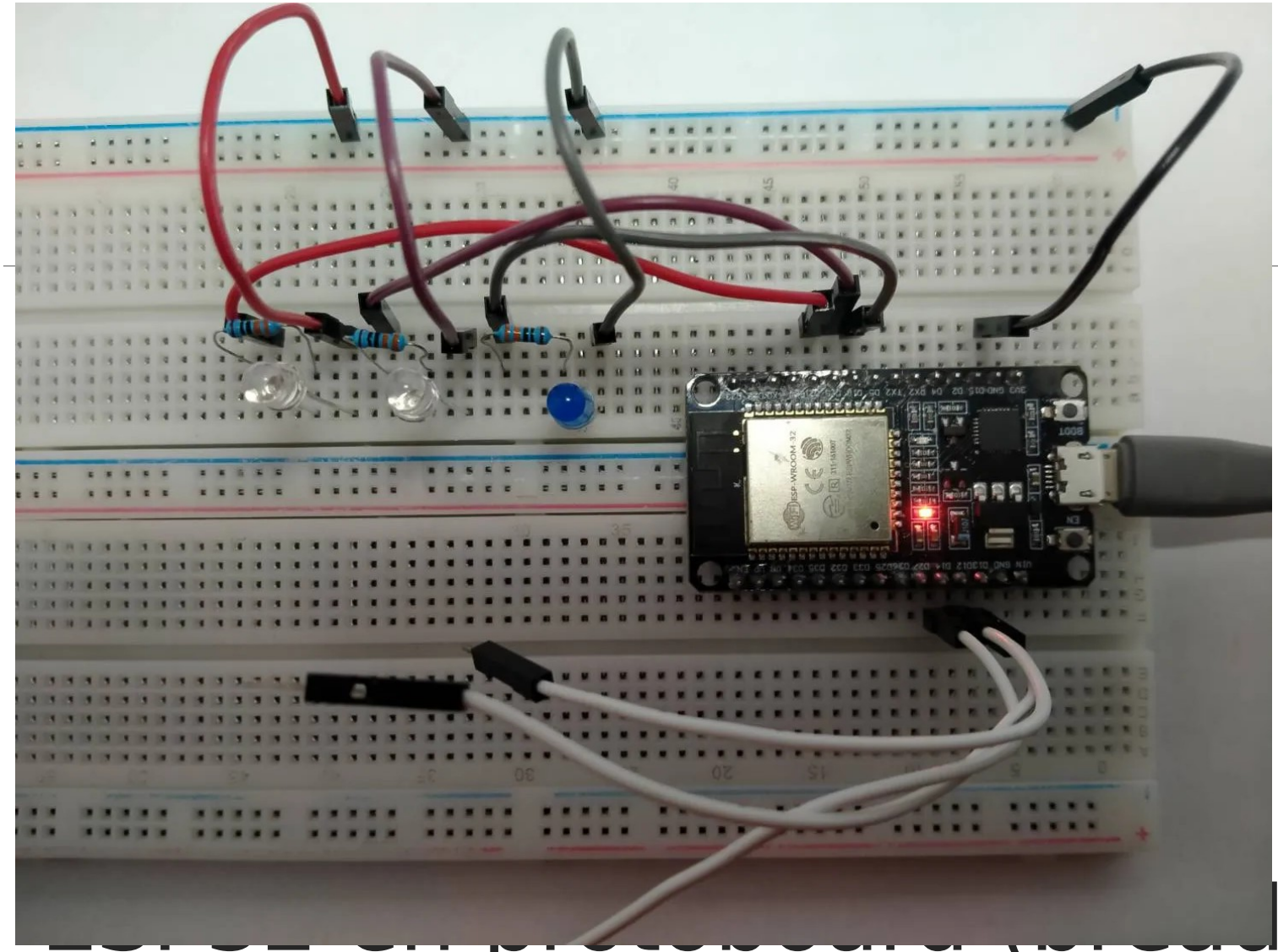
Hay pines dedicados específicamente a la comunicación, los TX/RX son puertos serie, usan protocolo RS232.

La comunicación serial (mediante el monitor serial de la IDE) así como el Led integrado suelen usarse para depuración de los sketchs.

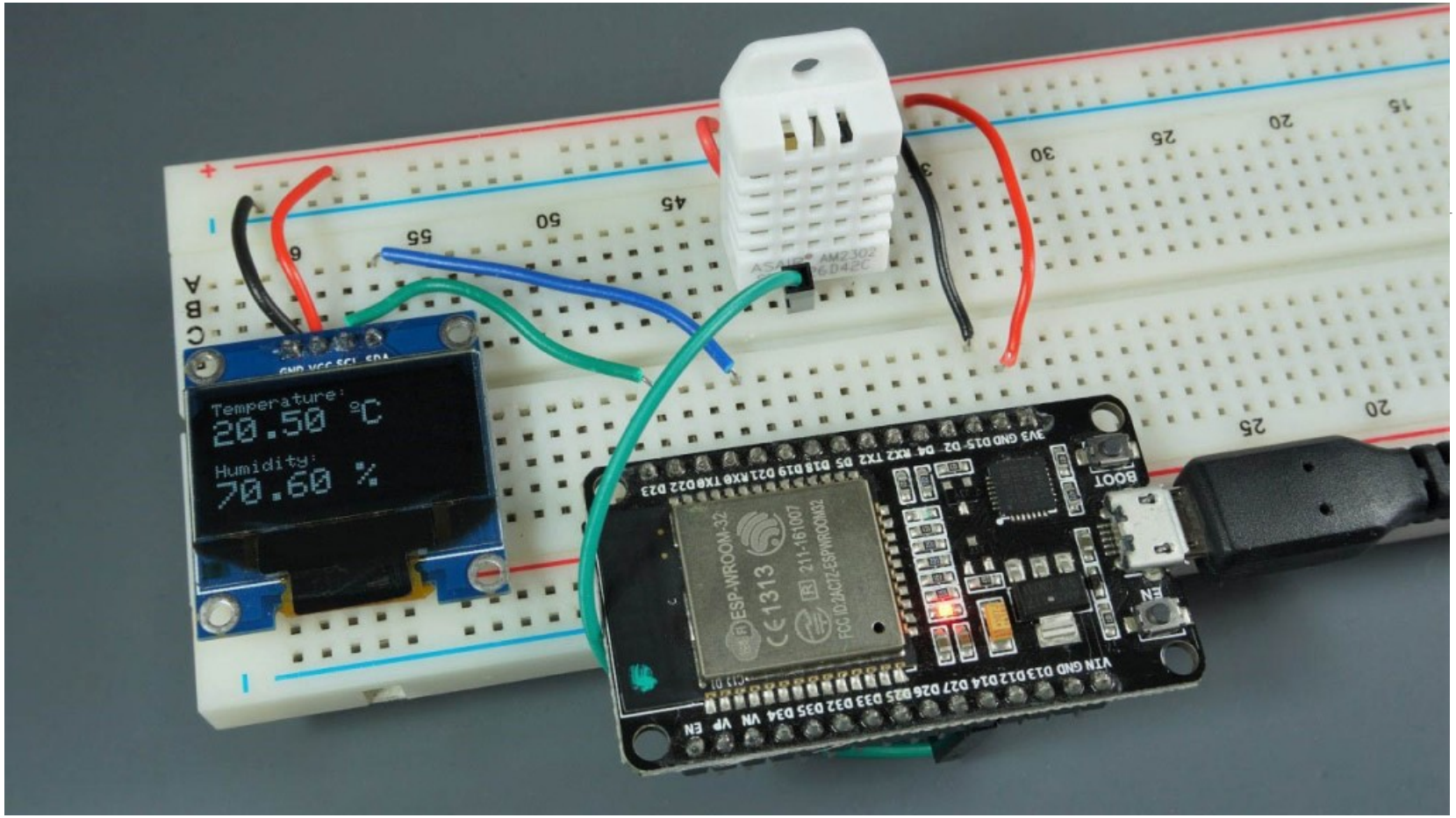
Existen otros pines que usan los protocolos SPI o I2C. Son importantes porque la mayoría de los periféricos tienden a usar uno de esos protocolos. Cada protocolo usa varios pines que se combinan para transmitir los mensajes. Tienen librerías que resuelven su utilización. Por ejemplo, para incorporar un display por I2C, debemos conectar GND, la alimentación, el pin SCL y el SDA con los respectivos de la placa.

(Cuando usamos Bluetooth y Wi-fi hay que evitar usar los pines marcados con ADC2, porque la ESP los usa para comunicarse con el modulo de Wi-Fi interno)

RTC (Real-Time Clock) sirven para “despertar” el micro desde un modo “sueño profundo”.



board)



ESP32 en protoboard (bread board)

¿Qué se necesita para programar la ESP32 DOIT KIT??

- IDE para Arduino
- VS Code + PlatformIO
- Drivers
- Cable USB (datos)
- Escribimos el código en **C++**
- Cargamos el programa (sketch) al micro



RTOS

Un RTOS (Real Time Operating System) es un SO cuyas tareas no pueden ser interrumpidas.

El RTOS coloca una capa que permite realizar interacción entre los núcleos.

(Debemos saberlo pero no lo explotaremos en este curso.)

